1: Executive Overview

Note: Dashboard uses 'B' for Thousands and 'M' for Millions

The Goal (The Story): This page is the "big picture" of the GardropAssistant app. It is designed for managers and investors. It answers a very simple question: "Who are our users, and how much money is sitting in their closets?"

1.1 Top KPI Cards

These numbers show the total size and health of our platform at a single glance.

Avg. Cost Per Wear: A great metric for sustainability. It shows if users are actually getting good value out of the clothes they buy.

> **How I did it (DAX):** I divided the total price of all clothes by the total number of times they were worn.

Avg Cost Per Wear = DIVIDE(SUM('garments'[price]), SUM('garments'[wearCount]), 0)

Total Items & Total Wardrobe Value: Shows the total number of clothes registered in the app and their total financial value. This proves the massive market size to investors.

How I did it (DAX): Count all rows in “Garments” and Sum of all “Garments Price”

Total Garments = COUNTROWS(garments)

Total Wardrobe Value = SUM(garments[price])

Active Users & Gender Breakdown: Shows how many unique users we have and their gender ratio using a clean Donut Chart.

How I did it (DAX): Count all rows in “Users”

Total Users = COUNTROWS(users)

1.2 Brand Investment Matrix (Scatter Chart)

This chart shows where our users spend their money.

The Story: Brands on the top left (like Prada, Gucci) are "Luxury", users buy fewer items but pay a high average price. Brands on the bottom right (like Zara, H&M) are "Fast Fashion", users buy many items for a lower price.

How I did it: X-Axis: Total Clothes (Count)

Y-Axis: Average Price

Values: Brand Name

1.3 Style Mix by Generation (100% Stacked Bar Chart)

This visual shows what different age groups like to wear.

The Story: Do Gen Z users prefer "Streetwear"? Do Gen X users own more "Formal" clothes? This helps the marketing and product teams understand their audience better.

How I did it: I calculated the users' ages using their birth dates in DAX, then grouped them into Gen Z, Gen Y, and Gen X using an “Switch” statement.

```
Generation =
SWITCH(
   TRUE(),
 users[age] <= 25, "Gen Z (18-25)",
```

```
    users[age] <= 40, "Gen Y (26-40)",
    "Gen X (41+)")
```

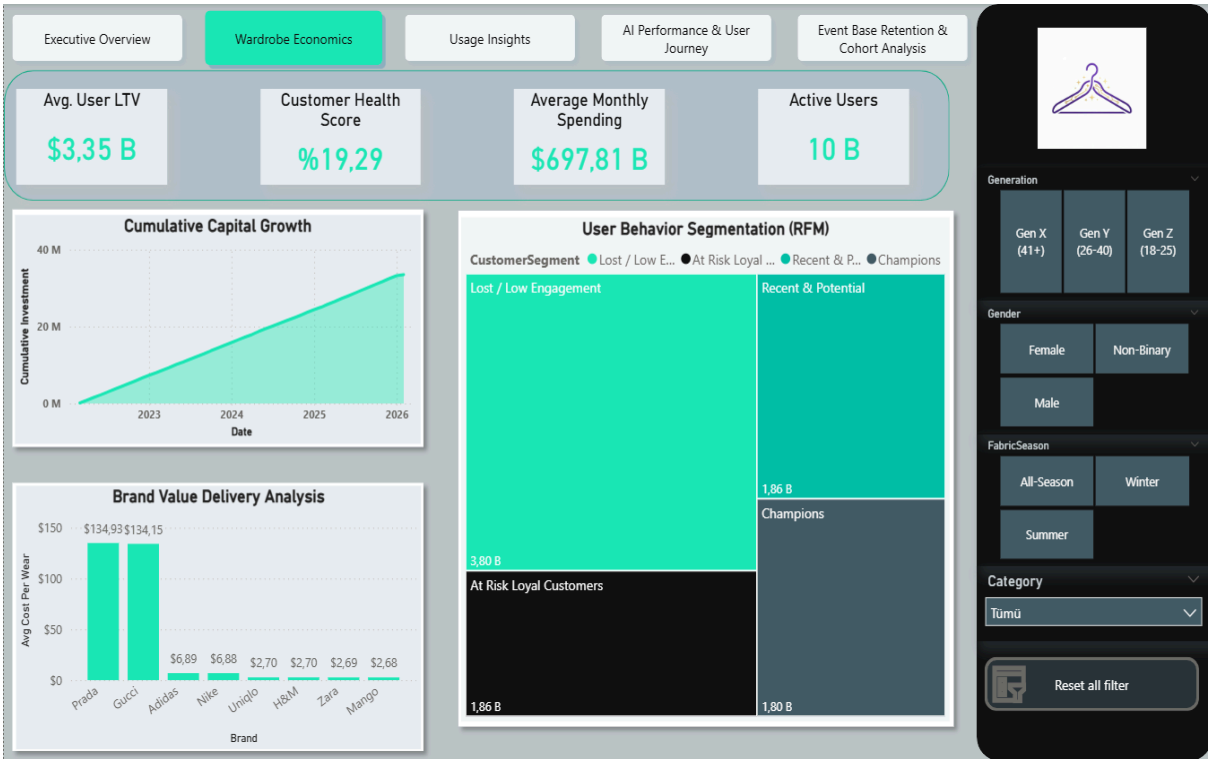
1.4 Category Based Total Items (Top 5 Donut Chart)

This shows the most popular clothing categories in our users' closets.

> **The Story:** What do our users own the most? (e.g., Jeans, Skirts). Our AI needs to know this to generate realistic daily outfits.
>
> **How I did it:** I used a "Top N" filter in the Power BI visual settings to only show the top 5 categories.

2: Wardrobe Economics

The Goal (The Story): If the first page was about "Who are our users?", this page is about "How do they spend, and how loyal are they?". This dashboard dives deep into the financial side of the users' closets and segments them based on their actual behavior (using RFM analysis).



2.1 Top KPI Cards (Financial & Health Metrics)

These cards track the financial pulse of the platform and the health of the user base.

Avg. User LTV (Lifetime Value): Shows the average total financial value a user brings to their digital closet.

How I did it: Divided the total value of all clothes by the total number of unique users.

Avg. User LTV = SUM(garments[price]) / COUNT(users[id])

Customer Health Score: A custom metric showing the percentage of "healthy/active" users. It gives managers a quick reality check on user retention.

How I did it:Firstly I created ”RFM Analysis View” in SQL and I linked this directly to the RFM analysis. Using DAX, I filtered our user base to only count the users who fell into our top RFM segments (like "Champions,Loyal Customers"). I then divided this number by the total user count.

RFM Analysis View(SQL):

```
CREATE VIEW vw_UserRFM_Analysis AS
    WITH UserStats AS (
     –Calculate Recency, Frequency, and Monetary metrics per user
    SELECT
     u.id AS userId,
    u.username,
    DATEDIFF(day, MAX(g.lastWornAt), GETDATE()) AS DaysSinceLastWear  -- Recency: Days since the last worn item,
    SUM(g.wearCount) AS TotalWears -- Frequency: Total number of times garments  were worn ,
    SUM(g.price) AS TotalSpent -- Monetary: Total value of the wardrobe
     FROM users u
    JOIN garments g ON u.id = g.userId
     GROUP BY u.id, u.username
    ),
```

```
RFM_Scores AS (
--Divide each metric into quartiles (1-4) using NTILE
SELECT
userId,
username,
DaysSinceLastWear,
TotalWears,
TotalSpent,
-- Recency is inversely proportional: fewer days get a higher score (4)
NTILE(4) OVER (ORDER BY DaysSinceLastWear DESC) AS R_Score,
NTILE(4) OVER (ORDER BY TotalWears ASC) AS F_Score,
NTILE(4) OVER (ORDER BY TotalSpent ASC) AS M_Score
FROM UserStats
WHERE TotalSpent > 0 AND TotalWears > 0
)
--Assign customer segments
SELECT
userId,
username,
R_Score, F_Score, M_Score,
CAST(R_Score AS VARCHAR) + CAST(F_Score AS VARCHAR) + CAST(M_Score AS
VARCHAR) AS RFM_Cell,
        CASE
WHEN R_Score >= 3 AND F_Score >= 3 AND M_Score >= 3 THEN 'Champions'
WHEN R_Score >= 3 AND F_Score <= 2 THEN 'Recent & Potential'
WHEN R_Score <= 2 AND F_Score >= 3 THEN 'At Risk Loyal Customers'
ELSE 'Lost / Low Engagement'
        END AS CustomerSegment
FROM RFM_Scores;
```

————————————————————————————————————

User Health Score(DAX) =

```
VAR TotalUsers = COUNTROWS('vw_UserRFM_Analysis')
VAR HealthyUsers =
CALCULATE(
COUNTROWS('vw_UserRFM_Analysis'),
'vw_UserRFM_Analysis'[CustomerSegment] IN {"Champions", "Loyal Customers"}
)
RETURN
DIVIDE(HealthyUsers, TotalUsers, 0)
```

Average Monthly Spending: Calculates the average value of new clothes added to the app per month.

How I did it (DAX): I took the total price of all clothes (garments) ever added to the app and divided it by the total number of unique months in our timeline.

Avg Monthly Spending(DAX) =

```
VAR TotalValue = SUM('garments'[price])
VAR TotalMonths = COUNTROWS(SUMMARIZE('garments', 'garments'[createdAt].[Yıl], 'garments'[createdAt].[Ay]))
RETURN
  DIVIDE(TotalValue, TotalMonths, 0)
```

Active Users: Total unique users in our “users” table.

2.2 Cumulative Capital Growth (Area Chart)

This chart shows the total money invested into users' closets over time.

The Story: Investors love to see "up and to the right" graphs. Instead of showing daily ups and downs, this running total proves that the platform's total managed value is constantly growing month by month.

How I did it (DAX): I used a cumulative sum (running total) pattern. It calculates the sum of all clothing prices from the beginning of time up to the current date on the axis.

Cumulative Investment =

```
CALCULATE(
  SUM('garments'[price]),
  FILTER(
    ALL('garments'),
    'garments'[createdAt] <= MAX('garments'[createdAt])
  ))
```

2.3 Brand Value Delivery Analysis (Column Chart)

This visual breaks down the "Average Cost Per Wear" by brand.

The Story: This is the ultimate wardrobe reality check. Luxury brands like Prada and Gucci cost users around $134 every time they wear them. Meanwhile, fast fashion brands like Zara and H&M cost less than $3 per wear. This tells the AI model: "If a user owns a Prada item, recommend it more often to help them lower their cost-per-wear and justify their investment!"

How I did it: I used the same “Avg Cost Per Wear” measure from Page 1, but put the “Brand” column on the X-axis to see the breakdown.

User Behavior Segmentation (RFM Treemap)

This is the most advanced analytical piece on this page. It groups users into specific buckets using **RFM Analysis (Recency, Frequency, Monetary)**.

The Story: Not all users are equal.

1. **Champions:** Users who use the app frequently and have valuable closets.
2. **At Risk Loyal Customers:** People who used to use the app a lot but haven't logged in recently. (Marketing needs to send them a push notification!)
3. **Lost / Low Engagement:** Users who signed up but barely use the AI.
4. **Recent & Potential:** New users who are just starting to build their digital wardrobe.

How I did it: I use “RFM Analysis View”s segments I calculated before and I simply dragged the “CustomerSegment” column into a Power BI Treemap visual as the "Category" and used the distinct count of “userId” as the "Values".

3: Usage Insights

The Goal (The Story): While previous pages tracked money and users, this page tracks context. It answers the core product questions: "Are users actually liking the outfits our AI suggests? And how do real-world factors like daily activities and weather influence their wardrobe choices?"

3.1 Top KPI Cards (Usage Pulse)

These cards highlight the most common behaviors and app usage patterns.

Most Worn Category Dynamic text cards showing what our users wear most (Jeans

How I did it (DAX): I used a combination of “TOPN” and “VALUES” to dynamically return the name of the category with the highest row count, rather than a number.

Most Worn Category =

```
VAR TopCategoryTable =
  TOPN(
    1,
    VALUES('garments'[category]),
    CALCULATE(SUM('garments'[wearCount])),
    DESC
  )
RETURN
  MAXX(TopCategoryTable, 'garments'[category])
```

Avg Item Per Outfit: Shows the complexity of the generated outfits. (e.g., A 3.45 average means users usually wear a top, bottom, shoes, and sometimes an accessory/jacket).

How I did it: Divided the total number of garments attached to events by the total number of distinct outfit events.

Avg Items per Outfit =

```
DIVIDE(
    COUNTROWS('outfit_items') ,
    DISTINCTCOUNT('outfits'[id]) ,
    0)
```

Top Event Type: Dynamic text card showing what our users do most (Casual Outing).

How I did it (DAX): I used a combination of “TOPN” and “VALUES” to dynamically return the name of the event with the highest row count, rather than a number.

Top Event Type =

```
VAR TopEventTable =
  TOPN(
    1,
    VALUES('events'[activityType]),
    CALCULATE(COUNTROWS('outfits')),
    DESC
  )
```

```
RETURN
    MAXX(TopEventTable, 'events'[activityType])
```

Distinct Event Count: Shows how many different event types we have in our customers' events

Event Count = DISTINCTCOUNT(events[activityType])

3.2 Generated Outfit Wear Rate (Pie Chart)

This is the ultimate measure of the AI's success (Conversion Rate).

The Story: When the app's AI suggests an outfit, does the user actually accept and wear it (True), or do they reject it (False)? A ~78% wear rate shows that the AI is doing a great job at matching user tastes.

How I did it: A clean Pie Chart using the boolean "worn" column (True/False) as the legend and the count of events as the values.

3.3 Outfit by Activity Type (Horizontal Bar Chart)

This visual shows exactly why users open the app in their daily lives.

The Story: Are people using our AI to dress up for fancy weddings, or just to go to the office? The chart clearly shows that "Daily Routine" and "Casual Outing" are the absolute winners.

How I did it: I used a Horizontal Bar Chart (putting "Activity Type" on the Y-axis) so the long category names aren't cut off. I put the "Outfit Count" on the X-axis and sorted it from highest to lowest to instantly highlight the most popular lifestyles.

3.4 Weather Impact on Wardrobe Usage (Heatmap Matrix)

This visual shows exactly how real-world weather conditions effect our users' digital wardrobe choices.

The Story: Since our app has a weather integration feature, I needed to prove that it actually works. Do users really listen to the AI and wear "Coats" when it's freezing (-10°C) and "Shorts" when it's 30°C? This heatmap acts as a reality check. It proves that user behavior in the app perfectly matches the real-world temperature outside.

How I did it: I grouped the raw weather data into clean "10-degree buckets" (-10, 0, 10, 20, 30). I created a Matrix visual with clothing “Category” on the rows and these “Temperature” buckets on the columns.

4: AI Performance & User Journey

The Goal (The Story): This dashboard answers a critical question: *"Is our AI actually smart, or is it frustrating the users?"* Instead of tracking money or demographics, this page tracks the "friction" between the user and the artificial intelligence. It measures how many times a user has to hit the "Regenerate" button before they are happy with their outfit.

4.1 Top KPI Cards (The AI Health Check)

These four metrics define the success or failure of our recommendation engine.

Avg AI Generated Outfit for an Event:

The Story: When a user asks for an outfit, how many times do they hit "Regenerate"? A score of 1.49 is excellent, it means users typically accept the first or second outfit the AI suggests. If this number was 5.0, it would mean our AI is doing a terrible job and frustrating users.

How I did it (DAX): I divided the total number of generated outfits by the total number of created events.

Avg AI Generated Outfit for an Event =

```
VAR OutfitCount = COUNTROWS(ValidOutfits)
VAR EventCount = CALCULATE(DISTINCTCOUNT('outfits'[eventId]), ValidOutfits)
RETURN
DIVIDE(OutfitCount, EventCount, 0)
```

First-Shot Success:

The Story: It shows that 65% of the time, the AI nails the perfect outfit on its very first try, requiring zero regenerations.

How I did it (DAX): I counted the events that have exactly 1 generated outfit associated with them, and divided it by the total number of events.

First-Shot Success =

```
VAR SingleShotEvents =
  FILTER(
    VALUES('outfits'[eventId]),
    CALCULATE(COUNTROWS('outfits')) = 1
  )
VAR SingleShotCount = COUNTROWS(SingleShotEvents)
VAR TotalEvents = COUNTROWS(VALUES('outfits'[eventId]))
RETURN
  DIVIDE(SingleShotCount, TotalEvents, 0)
```

Outfits Without Event:

The Story: Are users only using the app when they have a specific plan, or are they just playing with it? This metric shows that ~38% of outfits are generated purely for fun or inspiration, without being tied to a calendar event.

How I did it (DAX): I divided the count of outfits where the “eventId” is BLANK/NULL by the total number of generated outfits.

Spontaneous Usage =

```
VAR SpontaneousOutfits =
  CALCULATE(
    COUNTROWS('outfits'),
    ISBLANK('outfits'[eventId])
  )
VAR TotalOutfits = COUNTROWS('outfits')
RETURN
  DIVIDE(SpontaneousOutfits, TotalOutfits, 0)
```

Unstyled Events:

The Story: This is our main "Drop-off" or "Churn" metric. It shows that in 54% of created events, the user opened the app, created an event, but left without generating or saving an outfit.

How I did it (DAX): I counted the events that have “0” outfits attached to them, divided by total events.

Unstyled Events =

```
VAR RealEventsCreated =
  COUNTROWS('events')

VAR RealEventsWithOutfits =
  CALCULATE(
    DISTINCTCOUNT('outfits'[eventId]),
    NOT ISBLANK('outfits'[eventId])
  )
```

```
VAR Unstyled = RealEventsCreated - RealEventsWithOutfits
RETURN
    DIVIDE(Unstyled, RealEventsCreated, 0)
```

4.2 AI Friction Level (Horizontal Bar Chart)

The Story: This visualizes the "First-Shot Success" in detail. It proves that if the AI doesn't get it right in the first 1-2 attempts, the user usually gives up. The massive drop-off at the "High Friction (4+)" level shows that users have no patience for a struggling AI.

How I did it: I created a DAX Calculated Column to group events into text buckets based on their outfit count: "1. First-Shot", "2. Two Attempts", "3. Three Attempts", and "4. High Friction (4+)". I placed this on the Y-Axis and “Event Count” on the X-Axis.

AI_Outfit_Generations_Count =

```
CALCULATE(
COUNTROWS('outfits'),
'outfits'[eventId] = EARLIER('events'[id]))
```

AI_Friction_Level =

```
SWITCH(
TRUE(),
 'events'[AI_Outfit_Generations_Count] = 1, "1. First-Shot",
 'events'[AI_Outfit_Generations_Count] = 2, "2. Two Attempts",
 'events'[AI_Outfit_Generations_Count] = 3, "3. Three Attempts",
'events'[AI_Outfit_Generations_Count] >= 4, "4. High Friction (4+)",)
```

4.3 AI Struggle Area by Activity Type (Column Chart)

The Story: Does the AI struggle more with formal wear or gym clothes? This chart shows the “Avg AI Generated Outfit” broken down by activity. The friction is almost perfectly balanced (around 1.49 - 1.50) across all categories.

How I did it: I put “Activity Type” on the X-Axis and my “Avg AI Generated Outfit for an Event” measure on the Y-Axis.

Avg AI Generated Outfit for an Event =

```
VAR ValidOutfits =
  FILTER(
    'outfits',
    NOT ISBLANK('outfits'[eventId])
  )
VAR OutfitCount = COUNTROWS(ValidOutfits)
VAR EventCount = CALCULATE(DISTINCTCOUNT('outfits'[eventId]), ValidOutfits)
RETURN
  DIVIDE(OutfitCount, EventCount, 0)
```

4.4 Usage Type (Pie Chart)

The Story: A simple breakdown proving that the app is primarily used as a daily driver. ~61% of usage is for "Daily Routine" versus ~38% for "Planned Events".

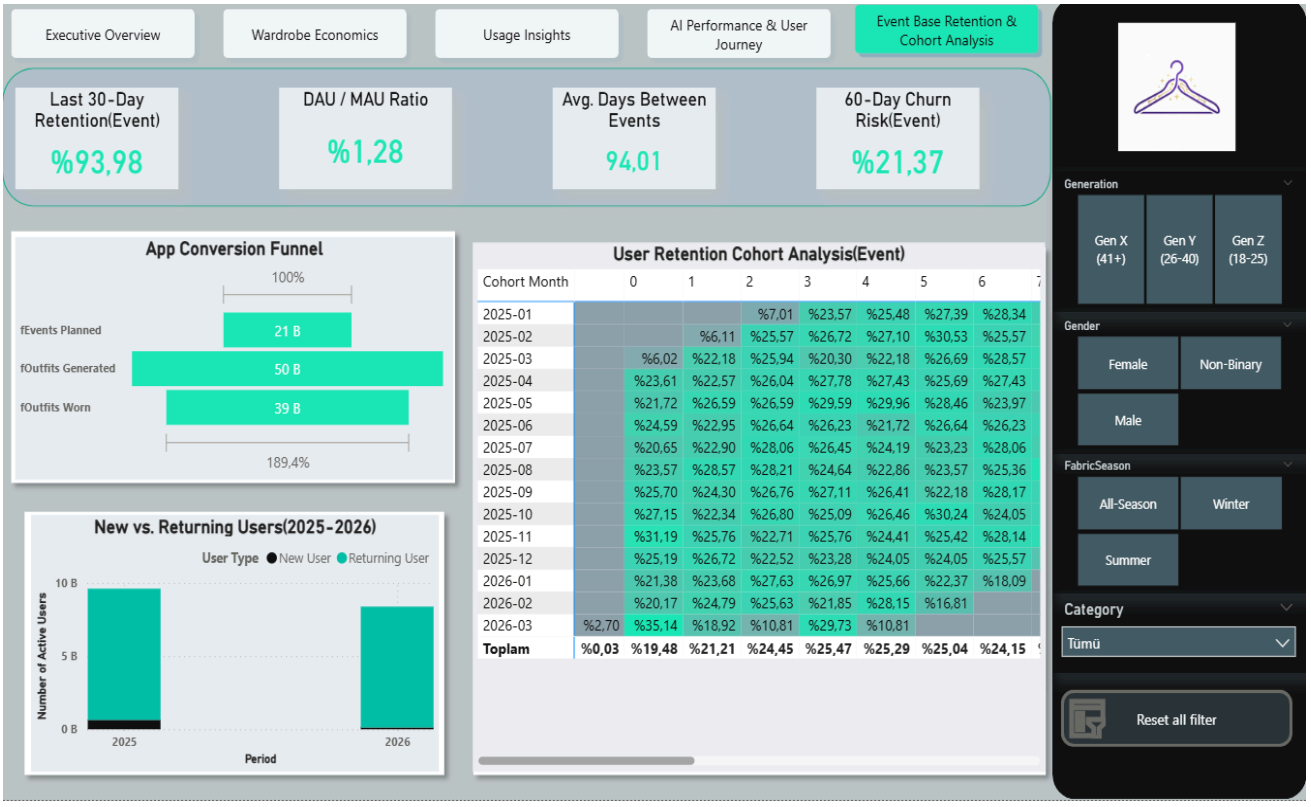
How I did it: I used a conditional grouping column (outfits have no eventID as Daily Routine, and others as Planned Events) as the legend, and the total event count as the values.

Usage Type =

```
IF(
  ISBLANK('outfits'[eventId]),
  "Daily Routine (Spontaneous)",
  "Planned Occasion"
)
```

5: Event Base Retention & Cohort Analysis

The Goal (The Story): This is the ultimate "Product Stickiness" dashboard. "Do users actually stay, or do they leave after downloading the app?" and "Where exactly are we losing them in the journey?"



5.1 Top KPI Cards (The Growth Pulse)

These metrics measure the true heartbeat of the application's user base.

Last 30-Day Retention :

The Story: A massive indicator of recent success. It shows that nearly 94% of our historically active user base has interacted with the app in the last 30 days.

How I did it(SQL): I calculated the distinct users who signed up and created one or more events in the last 30 days and divided it by the total distinct users who have created accounts in the last 30 days.

```
CREATE VIEW vw_KPI_Day30Retention AS
WITH CohortUsers AS(
SELECT COUNT(id) as TotalUserLast30Days
from users
where createdAt <= DATEADD(DAY, -30, GETDATE())
),
RetainedUsers AS (
SELECT COUNT(DISTINCT e.userId) AS RetainedCount
from users u
left join events e on u.id = e.userID
WHERE u.createdAt <= DATEADD(DAY, -30, GETDATE()) AND e.date >= DATEADD(DAY, 30, u.createdAt)
)
SELECT
   (CAST(r.RetainedCount as float) / NULLIF(c.TotalUserLast30Days,0)) as Day30EventRetentionRate
FROM CohortUsers c, RetainedUsers r
```

DAU / MAU Ratio:

The Story: Daily Active Users divided by Monthly Active Users. It tells us what percentage of our monthly users log in on any given day.

How I did it(SQL): DIVIDE(Average Daily Users, Average Monthly Users).

```
CREATE VIEW vw_KPI_Daily_Monthly_Ratio AS
WITH DailyActives AS (
   -- Distinct users per day over the last 30 days
   SELECT date, COUNT(DISTINCT userId) AS DAU
   FROM events
   WHERE date >= DATEADD(day, -30, GETDATE())
   GROUP BY date
),
AvgDAU AS (
   SELECT AVG(CAST(DAU AS FLOAT)) AS AvgDailyActive
   FROM DailyActives
```

```
),
MonthlyActives AS (
    SELECT COUNT(DISTINCT userId) AS MAU
    FROM events
    WHERE date >= DATEADD(day, -30, GETDATE())
)
SELECT
    a.AvgDailyActive / NULLIF(m.MAU, 0) AS DAUMAU_Ratio
FROM AvgDAU a, MonthlyActives m;
```

Avg. Days Between Events:

The Story: How often do users need an AI wardrobe assistant? This shows that users typically return to plan outfits roughly every 3 months (likely aligning with seasonal wardrobe changes. It is because our data is not real world data its dummy data created by AI).

How I did it: I used SQLto find the date difference between a user's current event and their previous event, then averaged it across the platform.

```
CREATE VIEW vw_KPI_AvgDaysBetweenEvents AS
WITH UserEventStats AS (
    SELECT
        userId,
        DATEDIFF(day, MIN(date), MAX(date)) AS ActiveDays,
        COUNT(id) - 1 AS Intervals
    FROM events
    GROUP BY userId
    HAVING COUNT(id) > 1 -- Only consider users with at least 2 events
)
SELECT
    AVG(CAST(ActiveDays AS FLOAT) / NULLIF(Intervals, 0)) AS AvgDaysBetweenEvents
FROM UserEventStats;
```

60-Day Churn Risk :

The Story: These are users who haven't generated an event in the last 60 days and are at high risk of deleting the app.

How I did it(SQL): Users that haven't created an event for the last 60 days divided by count of all users.

```
CREATE VIEW vw_KPI_EstimatedChurn AS
WITH UserStatus AS (
    SELECT
        u.id,
        MAX(e.date) AS LastEventDate
    FROM users u
    LEFT JOIN events e ON u.id = e.userId
    GROUP BY u.id
)
SELECT
    SUM(CASE WHEN LastEventDate < DATEADD(day, -60, GETDATE()) OR LastEventDate IS NULL THEN 1 ELSE 0 END) * 1.0
    / NULLIF(COUNT(id), 0) AS EstimatedChurnRate
FROM UserStatus;
```

5.2. App Conversion Funnel

This visualizes the user journey from intention to actual real-world action.

The Story: Unlike a standard sales funnel that strictly shrinks, ours has a unique shape. A user plans an event (Events Planned), the AI gives them multiple outfit options (Outfits Generated is higher than events), and finally, they select and wear one (Outfits Worn). Tracking the drop-off from "Generated" to "Worn" helps us measure the AI's real-world acceptance rate.

How I did it: I used the standard Funnel visual and placed three distinct measure calculations (Total Events, Total Generated Outfits, Total Worn Outfits) into the Values field to build the stages.

5.3 New vs. Returning Users (Stacked Column Chart)

Shows the compounding health of the user base over time.

The Story: A healthy application should have a massive, growing base of "Returning" users compared to "New" users. This chart proves that our application retains its users year over year, rather than just constantly replacing churned users with new marketing acquisitions.

How I did it: I created a DAX Calculated Column in the Events table. If the month of the event matches the user's registration month, they are labeled "New User". If the event date is greater than their registration date, they are labeled "Returning User".

User Type =

```
VAR UserRegistrationDate =
  LOOKUPVALUE(
    'users'[createdAt],
    'users'[id],
    'events'[userId]
  )
RETURN
IF(
  ISBLANK('events'[date]) || ISBLANK(UserRegistrationDate),
  BLANK(),
  IF(
    YEAR('events'[date]) = YEAR(UserRegistrationDate) &&
    MONTH('events'[date]) = MONTH(UserRegistrationDate),
    "New User",
    "Returning User"  ))
```

5.4 User Retention Cohort Analysis (The Heatmap)

Tracks long-term user loyalty and app stickiness.

The Story: If 100 users download the app in January, how many are still generating outfits 3 months later? This heatmap answers that. By reading left to right, we see if the app is becoming a habit. Dark green shows strong retention; light colors highlight drop-offs.

How I did it: I used a clean, 3-step approach to optimize performance:

1. **Cohort Month:** Grouped users by their sign-up month.
2. **Month Index:** Calculated the months passed between sign-up and each event (Month 0, 1, 2...).
3. **Cohort Retention %:** Divided active users in Month X by the total original users in that cohort.